

Assignment 3 | Bridge Pattern

Name: Amangeldi Arailym

Group: SE-2528

Topic: A - Drawing shapes

Repository: https://github.com/araildiil/Assignment3_SE-2528_Amangeldi_Arailym.git

Submitted commit hash: b870a537efe8b3846b39f9adc0fe34954ca7e369

1. Two independent dimensions and why Bridge fits

The application has two orthogonal axes of variation, and each one changes for its own reasons.

The Shape dimension (abstraction). This axis answers what is drawn. It varies in the kind of geometric figure and in the data that figure needs:

- Circle carries a radius of 2 and knows how to call a circle-rendering operation.
- Square carries a side of 3 and knows how to call a square-rendering operation.

Adding a new shape means adding one new subclass of Shape. Nothing on the renderer side has to change.

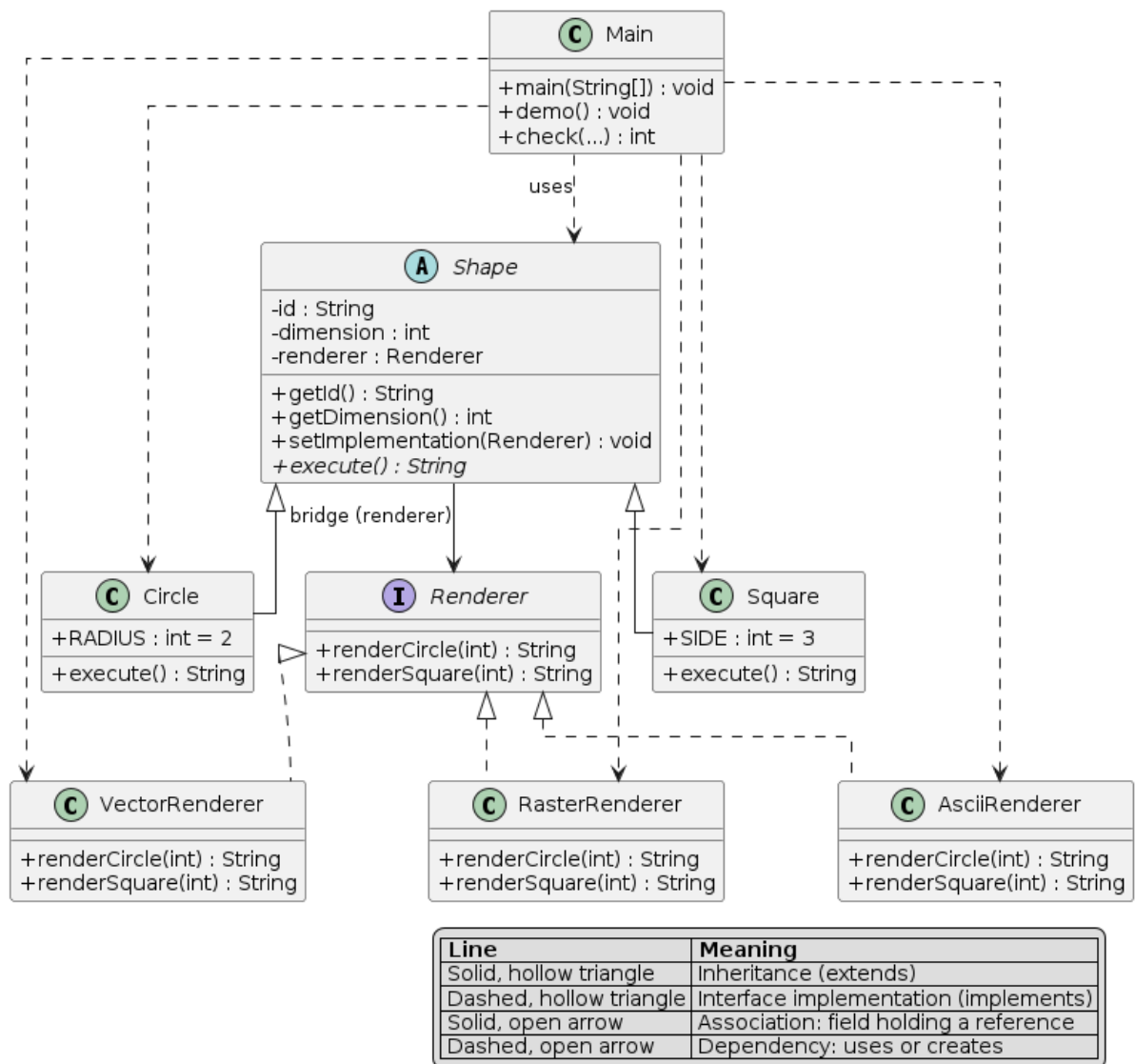
The Renderer dimension (implementation). This axis answers how the drawing is produced. It varies in the low-level output format:

- VectorRenderer produces a vector-style description.
- RasterRenderer produces a raster-style description.
- AsciiRenderer produces an ASCII-style description.

Adding a new renderer means adding one new implementation of the Renderer interface. Nothing on the shape side has to change.

Why subclassing every combination would not scale. Without Bridge, each combination would have to be its own class: VectorCircle, RasterCircle, AsciiCircle, VectorSquare, RasterSquare, AsciiSquare. That is two shapes $\times$ three renderers = six classes for the current state, and the count grows as a product. Every new shape multiplies the whole renderer set, and every new renderer multiplies the whole shape set. Adding a single triangle would immediately require three more classes; adding a fourth renderer would require four more. Bridge replaces this product with a sum by keeping the two hierarchies separate and connecting them through one interface-typed reference. In our code that reference is the field `renderer` in Shape, supplied through the constructor and replaceable at runtime via `setImplementation(...)`.

2. UML class diagram



3. Five annotated code excerpts

(a) Separation of responsibilities

```

public abstract class Shape {
    private final String id;
    private final int dimension;
    private Renderer renderer;

    public abstract String execute();
}
  
```

Shape stores only domain data and an interface reference. It never mentions VectorRenderer, RasterRenderer, or AsciiRenderer, and it never branches on their concrete types. The

rendering decision lives entirely on the other side of the bridge, so the two hierarchies can be read, tested, and modified independently.

(b) Meaningful names

```
public static final int RADIUS = 2;
public static final int SIDE = 3;

public void setImplementation(Renderer renderer) { ... }
public String execute() { ... }
```

Names such as RADIUS, SIDE, setImplementation, execute, renderCircle, and renderSquare say exactly what the code does. There are no abbreviations like r, no generic names like data or doIt, and no ambiguity about which side of the bridge a method belongs to.

(c) Small focused methods

```
@Override
public String execute() {
    return getRenderer().renderCircle(getDimension());
}
```

Each execute() does exactly one thing: delegate to the correct low-level operation on the stored renderer. Small methods like this are easy to test, easy to override in a new shape, and impossible to accidentally duplicate.

(d) Avoidance of duplication

```
static int check(String id, boolean ok, String classes, String actual, String expected) {
    if (ok) {
        System.out.println(id + " PASS | " + classes + " | result=" + actual);
        return 1;
    }
    System.out.println(id + " FAIL | " + classes + " | actual=" + actual
        + " | expected=" + expected);
    return 0;
}
```

The PASS/FAIL printing logic exists in exactly one place and is reused by all seven demo checks (T1-T7). Rendering formats live only inside their own renderer classes, and shape-specific data lives only in the corresponding shape. No workflow or formatting logic is duplicated across files.

(e) Encapsulation of state

```
private final String id;
private final int dimension;
private Renderer renderer;
```

```
public void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer);
}
```

id and dimension are private final, so a shape cannot silently change its identity or its size after construction. The renderer field is private and can only be replaced through setImplementation(...), which rejects null. Subclasses read state via getDimension() and getRenderer(), so the internal representation is never exposed.

4. Extension and comparison

How AsciiRenderer was added

The extension introduced one new class implementing the existing Renderer interface:

```
public class AsciiRenderer implements Renderer {
    @Override
    public String renderCircle(int radius) { return "ASCII (circle r=" + radius + ")"; }
    @Override
    public String renderSquare(int side) { return "ASCII [square s=" + side + "]"; }
}
```

Two new checks (T6 and T7) were added to Main.demo(), and the class was registered in sources.txt.

What stayed unchanged

Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer were not modified. This is verifiable from extension.diff: it contains only the new src/AsciiRenderer.java and the additions inside src/Main.java. No existing abstraction, interface, or implementation was touched - which is exactly the promise of Bridge.

One specific trade-off

Bridge introduces an extra level of indirection. Compared with a single class hierarchy, the code now has one more interface, one more field per abstraction object, and a call chain Main → Shape.execute() → Circle.execute() → Renderer.renderCircle() before any actual work is done. There are also more types to maintain (Shape + two subclasses + Renderer + three implementations = seven classes instead of two or three). For a problem with only one axis of variation this would be overkill; Bridge pays off only when both dimensions genuinely vary independently, as they do here.

How Bridge differs from Adapter in intent

Adapter is a post-hoc compatibility fix. It wraps an existing class whose interface does not match what the client expects, and translates calls so the client can use that class without

modifying it. The wrapped class and the target interface already existed before the adapter was written.

Bridge is a design-time separation. In our project, `Renderer` is not adapting some pre-existing, incompatible class so that `Shape` can use it. It is deliberately declared as the low-level contract that any drawing implementation must follow, so that `Shape` can delegate to any of them and switch at runtime (T5). An adapter would not support that runtime replacement - it would just expose the old class under a new name. Bridge's intent is to let the two hierarchies evolve independently; Adapter's intent is to make two already-fixed interfaces work together.

5. References

Lecture 4 - Bridge Pattern, course slides, 2026-2027.

E. Freeman, B. Freeman, K. Sierra, Head First Design Patterns, O'Reilly Media - Bridge and Adapter chapters.